

AI 产品经理作品附件

ReproPilot

面向机器学习论文复现的
代码执行与修复智能体

独立完成产品定义、Agent
状态机、工程实现、评测体系、用户研究与原型迭代

yihanx082-cmd · 2026.09 · 作品集级 MVP

为什么“跑起来”仍不等于“复现成功”

论文复现横跨论文、代码、环境和实验协议。真正的风险不只是报错，还包括程序正常结束却比较了错误的数据、配置或指标。

“最危险的不是程序报错，而是程序正常结束，但实验协议已经和论文不一样了。”

计算机视觉方向硕士生，P02

用户任务	常见断点	误判风险
搭建环境	旧版 PyTorch、CUDA、驱动、依赖缺失	把安装问题当作论文或模型问题
对齐实验	学习率、数据划分、增强、checkpoint 版本	代码可运行，但协议已经偏离论文
比较结果	accuracy/error、macro/micro-F1、单次最好值	数字接近，却不能直接比较
判断成功	日志、diff、配置和结果散落在多处	缺少可追溯证据，结论依赖主观判断

产品机会

把分散的复现动作组织成一条可审查的证据链：系统不仅执行，还要解释做了什么、为什么修改、证据在哪里，以及当前结论能支持到什么程度。

目标用户与第一轮真实研究

目标用户不是完全不会编程的人，而是有基础
Python/机器学习经验、需要减少环境和排错成本，同时必须保留科学判断权的复现者。

5/5 完成核心流程	3/5 无帮助独立完成	2/5 首次误认可信度	3.8/5 满意度均值
---------------	----------------	----------------	----------------

研究发现

- 环境、依赖和数据版本仍是最显性的时间成本。
- 指标口径与 checkpoint 来源会产生更隐蔽、更高风险的错误。
- 新手容易把“代码跑通”“结果接近”和“复现可信”混成一个结论。
- 专业用户需要精确 diff、补丁哈希、测试断言与原始日志，不能只看绿色状态。

研究边界

样本为 5 人便利抽样，只用于发现问题和验证交互；第一轮未达到预设的 4/5 独立完成门槛，因此结论是“核心路径可完成，但可用性验证未通过”。没有逐任务计时和统一人工基线，所以不声称节省了多少时间或步骤。

产品闭环：从 PDF 到可信度报告

MVP 将通用大模型能力约束在论文抽取和补丁建议两个语义环节，其余关键控制由确定性程序完成。

01	论文证据解析	抽取实验设置、指标和论文结果，并保留页码证据
02	代码配置审计	扫描配置、README 和参数默认值，输出论文与代码的差异
03	隔离执行与修复	Docker 构建、最小运行、诊断、最小补丁、测试与回滚
04	正式实验与对比	记录多种子命令和日志，比较论文值与观测值
05	可信度报告	七维评分，每个得分绑定 artifact，明确证据边界

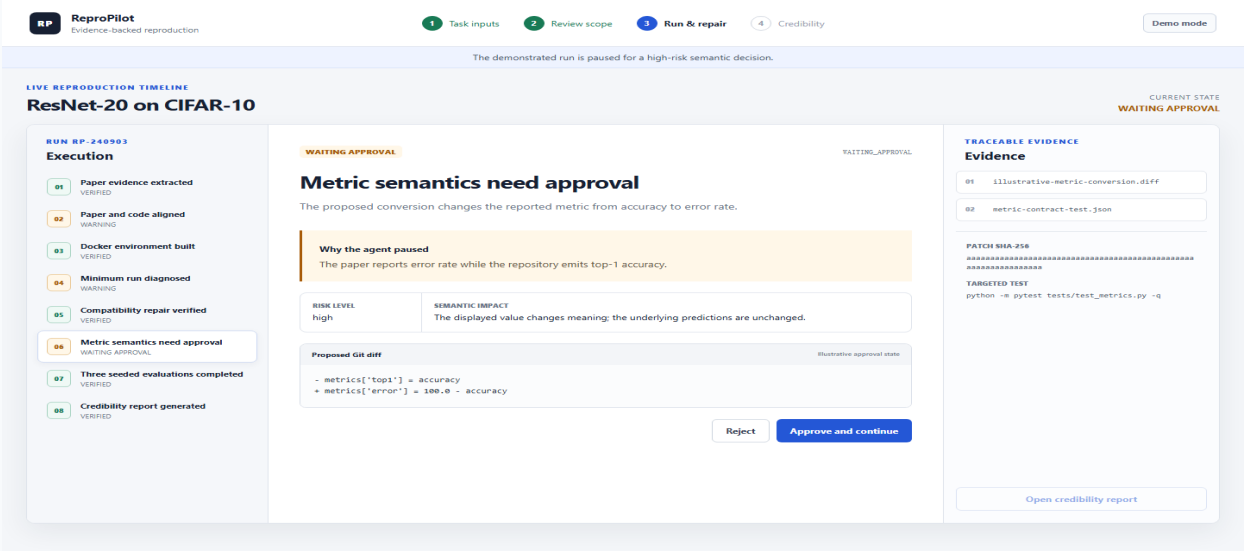
MVP 取舍

选择 Python CLI + 本地 Web + 冻结公开原型：CLI 负责真实执行，Web 解释状态和审批，公开原型保证面试演示稳定。暂不做账号系统、云 GPU、多用户协作或 Kubernetes，避免作品集 MVP 失焦。

关键产品机制：自动修复不能越过科学语义

系统区分“低风险兼容性修复”和“会改变实验含义的高风险修改”。后者必须暂停，让使用者基于完整证据作出决定。

机制	实现	用户价值
最小补丁	限制相关文件、最多 3 个文件/120 行	减少无关修改，便于审查
风险门	数据、指标、模型、权重修改必须审批	防止 Agent 静默改变实验语义
证据审批	根因、diff、数值示例、断言、退出码、日志	让批准基于证据，而非绿色按钮
SHA 绑定	审批绑定补丁 SHA-256	补丁变化后旧批准自动失效
事务回滚	验证失败后反向应用并校验恢复	避免工作副本被失败修复污染



原型执行时间线：阶段、暂停原因、补丁与测试证据在同一上下文中展示。

真实案例：结果接近，但仍只能判定 PARTIAL

ResNet-20/CIFAR-10 案例验证了端到端闭环，也展示了产品如何拒绝夸大结论。

8.75% 论文 Error	8.27% 观测 Error	-0.48pp 与论文差值	80/100 证据可信度
-------------------	-------------------	------------------	-----------------

运行发生了什么

- 从论文 PDF 中定位 ResNet-20/CIFAR-10 表格结果。
- 旧仓库在 CPU Docker 中因无条件调用 CUDA 首次失败。
- 第一版补丁暴露 checkpoint 缺少 epoch 元数据，系统先回滚再生成完整补丁。
- 目标测试通过后执行种子 11、22、33 的三个独立评估进程。

为什么不是完整复现

三个种子评估的是同一个公开 checkpoint，没有从头训练；原始数据划分 provenance、训练增强和 checkpoint 版本证据不完整。零方差只说明评估确定性，不能证明训练过程稳定。因此最终报告为 80/100 PARTIAL，而不是“论文已复现”。

评分维度	得分	证据边界
环境	15/15	Docker 构建证据完整
数据	10/20	数据可用，原始划分来源不完整
配置	10/20	只恢复了部分训练配方
指标	15/15	论文与运行 Error 可直接比较
随机种子	10/10	三个独立命令 artifact
结果接近	15/15	差值 0.48 个百分点
外部依赖	5/5	构建依赖可获得

Agent 评测：用故障注入测定位、修复与副作用

评测对象不是论文指标，而是 Agent
在固定真实仓库上面对未知故障时，能否在受控范围内持续定位、修复并验证。

3 模型实测仓库	9/9 反馈闭环完成	+100pp 完整任务提升	0 无关修改行
-------------	---------------	------------------	------------

单轮完整任务		0/9
反馈闭环完整任务		9/9
单轮阶段解决		9/18
反馈闭环阶段解决		18/18

故障类型

缺少依赖、错误数据路径、学习率不一致、Top-1 指标错误、CUDA 回退、验证集 shuffle。每个案例固定仓库提交、允许修改文件、故障探针与修复后语义探针。

证据边界

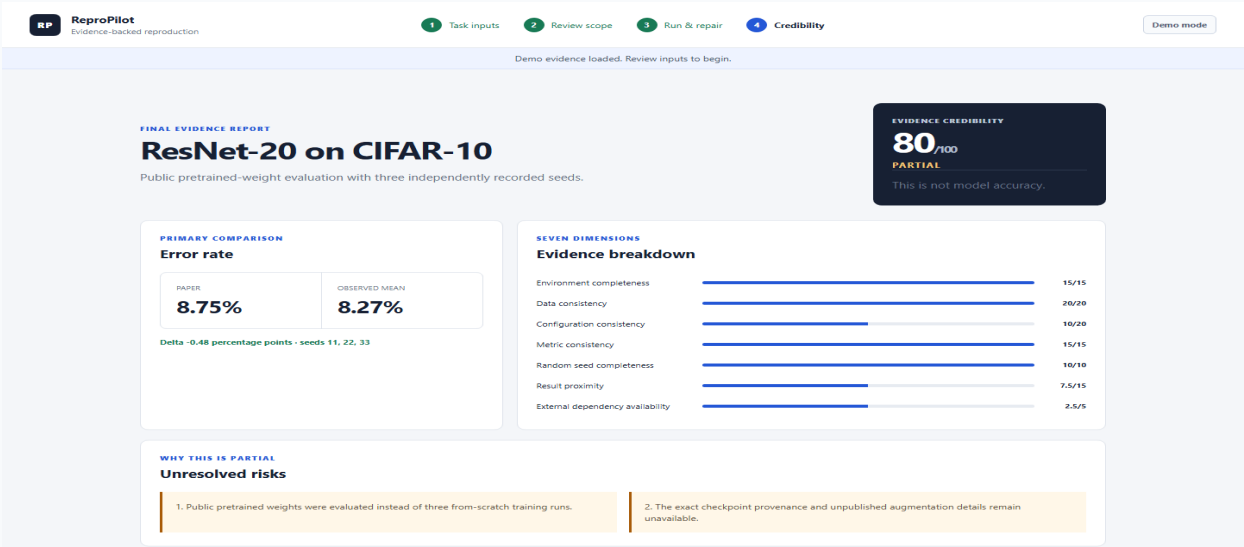
三重复双故障消融中，原始单轮完成 0/9、解决 9/18 个阶段；反馈循环完成 9/9、解决 18/18。单轮只有一次调用且第二个故障在首次修复后才暴露，因此 +100 个百分点证明闭环完成多阶段任务的价值，不能外推为任意代码修复任务的通用提升。记忆组也是 9/9，但本轮记忆命中为 0，不能归因于记忆机制。

运行成本	观测值
实验规模	4 组 × 3 次重复 × 3 个仓库
四组模型调用	55 次
四组 Token	共 180,202
反馈循环	18 次调用 / 60,047 tokens
无关修改	四组均为 0 行
模型美元成本	服务商未返回价格，保持未知

用户研究驱动的第二版，不隐藏第一次失败

第一轮测试暴露的最大问题不是功能缺失，而是状态和证据表达会让经验较少的用户产生错误结论。

观察证据	产品风险	第二版修改
2/5 把 80/100 当准确率	把证据评分误解成模型性能	永久显示“证据可信度，非模型准确率”
审批只看文件名、SHA 和命令	用户无法判断补丁是否真的正确	增加转换示例、论文引用、断言、退出码和日志
拒绝后仍显示 VERIFIED	状态与用户决定冲突	拒绝后进入 STOPPED，后续阶段全部停止
初学者不懂 diff/SHA	看见证据但无法使用证据	增加中文解释，同时保留原始技术材料



第二版可信度报告：分数与模型准确率长期分离，扣分原因可追溯到 evidence artifact。

下一轮验证标准

至少 4/5 无帮助完成流程、至少 4/5 正确解释 PARTIAL、0/5

将可信度分数误认为模型准确率。第二轮必须由真实参与者完成，AI 不能代替。

产品定位：不是更会聊天，而是更会管理证据

ReproPilot 不替代论文发现、托管环境或通用 Coding Agent；它把三者之间缺失的论文复现判断做成一条受控 workflow。

方案	擅长什么	ReproPilot 的差异
人工 workflow	灵活，能处理复杂科学判断	将重复执行与证据整理结构化，但保留人的判断权
Papers with Code	发现论文、代码、数据集和榜单	继续完成环境执行、修复与一次具体运行的审计
Code Ocean	封装、分享和重复运行计算环境	接手可能损坏的旧仓库，审计论文一致性并受控修复
通用 Coding Agent	完成通用仓库任务与 PR	增加论文声明、实验协议、科学指标与可信度状态机

窄定位

面向已有论文与仓库，从配置审计开始，经过隔离执行和受控修复，最终输出论文结果对比与可追溯的复现可信度判断。

为什么选择作品集级 MVP

- 先聚焦 PyTorch 图像分类与 10-20 分钟缩小实验，控制技术范围。
- 公开网页使用冻结证据，保证面试演示稳定；本地 live 模式连接真实状态机。
- 不承诺云端多租户、完整 GPU 调度和受限数据许可管理。

独立完成：从产品问题到可运行系统

项目不是单一原型或概念 PRD，而是一套产品判断、工程证据和用户验证互相约束的作品。

角色	完成内容
产品	问题定义、用户分层、竞品分析、PRD、MVP 边界、指标方案和迭代优先级
Agent/工程	状态机、PDF 证据抽取、代码审计、Docker 沙箱、诊断、补丁策略、Git 回滚、正式实验和报告
评测	真实仓库故障注入、语义探针、四组 Agent 消融、定位/修复/副作用/成本指标、证据边界
设计/研究	React 可点击原型、5 位真实参与者访谈与观察、问题归纳、第二版交互修复
交付	GitHub、CI、公开演示站、166 项 Python 测试（含 4 项 Docker 集成案例）、前端与 Worker 测试

技术栈

Python · Pydantic · Docker · Git · pytest · React · Vite · OpenAI 兼容 API · GitHub Actions

我学到的产品判断

Coding Agent 的价值不只是自动化更多步骤，而是在自动化过程中明确什么可以由系统决定、什么必须留给用户，以及每个结论需要哪些证据。对科研场景而言，正确表达不确定性本身就是产品能力。

当前完成度与下一步

工程 MVP、公开原型、第一轮用户研究、产品文档和求职表达已经形成闭环；剩余工作依赖新的真人或账号证据。

166 Python 测试	4 Docker 集成案例	5 真实参与者	4 Agent 消融组
------------------	------------------	------------	----------------

必须继续验证的四件事

- 第二轮真人复测：验证第二版是否消除可信度与状态误解。
- 加入调用预算更接近的基线，并构造可产生有效跨仓库同类记忆命中的案例。
- 录制本人 3-5 分钟讲解视频，补齐个人表达证据。
- 未来加入至少一个从头训练案例，验证训练过程而非 checkpoint 评估。

GITHUB	github.com/yih anx082-cmd/ReproPilot
LIVE PROTOTYPE	repropilot-evidence-agent.yizhuliang42.chatgpt.site

一句话总结

ReproPilot 把“代码能运行”与“论文已复现”拆成两个不同问题，并用可执行状态机、补丁控制和证据评分帮助用户做出更可信的判断。